Git Cheatsheet — With Explanations

A practical reference organized by what you're trying to *do*, not just alphabetically. Each command includes a plain-English explanation of what's actually happening under the hood.

---

1. Setup & Configuration

```bash
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Sets your identity for all commits on this machine. Drop `--global` to set it per-repo instead.

```bash
git config --global init.defaultBranch main
```

Makes new repos default to `main` instead of `master`.

```bash
git config --list
```

Shows all current config settings (useful for checking what's already set).

---

2. Starting a Repository

```bash
git init
```

Turns the current folder into a Git repo. Creates a hidden `.git` folder that stores all history — nothing is tracked yet.

```bash
git clone <url>
```

Copies a remote repo (and its full history) onto your machine. This is how you get a local copy of something on GitHub/GitLab.

```bash
git clone <url> my-folder-name
```

Same as above, but names the local folder yourself instead of using the repo's default name.

---

3. The Core Loop: Status, Add, Commit

Mental model: Git has three areas — your **working directory** (files on disk), the **staging area** (what you've marked as "ready to save"), and the **repository** (permanent history). `add` moves files from working → staging. `commit` moves staging → history.

```bash
git status
```

Shows what's changed, what's staged, and what's untracked. Run this constantly — it's your compass.

```bash
git add <file>
git add .
git add -A
```

- `add <file>` stages one specific file
- `add .` stages everything in the current directory and below
- `add -A` stages everything in the whole repo, including deletions

```bash
git commit -m "message"
```

Saves the staged changes as a permanent snapshot with a message. The message should explain *why*, not just *what*.

```bash
git commit -am "message"
```

Shortcut: stages all *tracked* files (skips new/untracked ones) and commits in one step.

```bash
git commit --amend
```

Edits the most recent commit — lets you fix the message or add forgotten changes to it, instead of making a new commit. **Don't do this on commits you've already pushed and others may have pulled.**

---

4. Viewing History

```bash
git log
```

Full commit history, newest first, with author/date/message.

```bash
git log --oneline
```

Condensed: one line per commit. Good for a quick scan.

```bash
git log --oneline --graph --all
```

Adds a visual branch graph — extremely useful for understanding how branches diverged and merged.

```bash
git show <commit-hash>
```

Shows exactly what changed in one specific commit.

```bash
git diff
```

Shows unstaged changes — what's different between your working directory and the last commit.

```bash
git diff --staged
```

Shows staged changes — what will actually go into your next commit.

```bash
git diff branch1 branch2
```

Compares two branches directly.

---

5. Branching

Mental model: A branch is just a movable pointer to a commit. Creating one is cheap and instant — there's no reason to be afraid of branching often.

```bash
git branch
```

Lists all local branches; the current one is marked with `*`.

```bash
git branch <name>
```

Creates a new branch (but doesn't switch to it).

```bash
git switch <name>
```

Switches to an existing branch. (Modern replacement for `git checkout <name>`.)

```bash
git switch -c <name>
```

Creates a new branch **and** switches to it in one step. (Replaces `git checkout -b <name>`.)

```bash
```

```
git branch -d <name>
```

Deletes a branch — only if it's already merged (safe delete).

```bash
git branch -D <name>
```

Force-deletes a branch even if unmerged (destructive — the commits become unreachable unless referenced elsewhere).

```bash
git branch -m <old> <new>
```

Renames a branch.

---

6. Merging & Rebasing

Mental model: Both combine changes from one branch into another, but they rewrite history differently.

- **Merge** creates a new "merge commit" that ties two histories together — nothing is rewritten, but history gets a branching structure.
- **Rebase** replays your commits on top of another branch — history stays linear, but commit hashes change.

```bash
git merge <branch>
```

Merges `<branch>` into your **current** branch. Run this while checked into the branch you want to receive the changes.

```bash
git merge --no-ff <branch>
```

Forces a merge commit even if a "fast-forward" (no divergence) would've been possible — useful for keeping a clear record that a feature branch happened.

```bash
```

```
git rebase <branch>
```

Moves your current branch's commits to start from the tip of `<branch>` instead, replaying them one by one. Cleaner history, but **never rebase commits that have been pushed and shared** — it rewrites hashes and breaks things for collaborators.

```bash
git rebase -i HEAD~3
```

Interactive rebase on the last 3 commits — lets you reorder, squash, edit, or drop commits. Extremely useful for cleaning up messy commit history before a PR.

---

7. Handling Conflicts

When Git can't auto-merge, it pauses and marks the conflicting sections in the file like this:

```
<<<<<<< HEAD
your version
=======
their version
>>>>>>> branch-name
```

Resolution steps:

1. Open the file, manually edit it to keep what you want, delete the `<<<<<<<`, `=======`, `>>>>>>>` markers
2. `git add <file>` to mark it resolved
3. `git commit` (if merging) or `git rebase --continue` (if rebasing)

```bash
git merge --abort
```

Bails out of a merge entirely, returning to the state before you started.

```bash
git rebase --abort
```

Same, but for an in-progress rebase.

8. Working with Remotes

```bash
git remote -v
```

Lists remotes (usually just `origin`) and their URLs.

```bash
git remote add origin <url>
```

Connects your local repo to a remote — needed after `git init` if you want to push somewhere.

```bash
git fetch
```

Downloads new commits/branches from the remote, but does **not** merge them into your local branches. Safe to run anytime — it just updates your knowledge of the remote.

```bash
git pull
```

Equivalent to `fetch` + `merge` in one step — downloads and immediately merges into your current branch.

```bash
git pull --rebase
```

Same, but rebases your local commits on top instead of merging — keeps history linear.

```bash
git push
```

Uploads your commits to the remote.

```bash
```

```
git push -u origin <branch>
```

Pushes and sets up tracking, so future `git push`/`git pull` on this branch know where to go without specifying remote/branch again.

```bash
git push --force
```

Overwrites the remote branch with your local one, even if it means losing commits on the remote. **Dangerous on shared branches** — use `--force-with-lease` instead when possible, since it fails safely if someone else pushed in the meantime.

---

9. Undoing Things

This is the section people forget most — so a bit more detail here.

```bash
git restore <file>
```

Discards unstaged changes in a file, reverting it to the last commit. (Old equivalent: `git checkout -- <file>`.)

```bash
git restore --staged <file>
```

Unstages a file (removes it from staging) but keeps your edits in the working directory.

```bash
git reset --soft HEAD~1
```

Undoes the last commit, but keeps all changes staged. Good for "I want to redo this commit message/split it up."

```bash
git reset --mixed HEAD~1
```

(Default behavior of `git reset HEAD~1`.) Undoes the last commit, unstages the changes, but keeps them in your working directory.

```bash
git reset --hard HEAD~1
```

Undoes the last commit **and deletes the changes entirely**. Destructive — only use if you're sure you don't need those changes.

```bash
git revert <commit-hash>
```

Creates a **new** commit that undoes the changes from a specific commit — doesn't rewrite history. Safer than `reset` for anything already pushed/shared, since it doesn't erase the original commit.

```bash
git clean -fd
```

Deletes untracked files and directories from your working directory. Add `-n` first (`git clean -fdn`) to preview what would be deleted before actually doing it.

---

10. Stashing

Mental model: A temporary shelf for changes you're not ready to commit but need to set aside (e.g., to switch branches quickly).

```bash
git stash
```

Saves your uncommitted changes (staged + unstaged) and reverts your working directory to match the last commit.

```bash
git stash list
```

Shows all stashed sets of changes.

```bash
git stash pop
```

Re-applies the most recent stash and removes it from the stash list.

```bash
git stash apply
```

Re-applies the most recent stash but **keeps it in the list** (useful if you want to apply the same stash to multiple branches).

```bash
git stash drop
```

Deletes a stash without applying it.

```bash
git stash -u
```

Also stashes untracked files (normally skipped).

---

11. Inspecting & Finding Things

```bash
git blame <file>
```

Shows who last modified each line of a file, and in which commit — good for tracking down "why is this line here."

```bash
git log --follow <file>
```

Shows the commit history of a file, even across renames.

```bash
git log -p <file>
```

Shows the full diff for every commit that touched a file.

```bash
git grep "search term"
```

Searches tracked files in the repo for a string — faster and Git-aware compared to a plain `grep -r`.

```bash
git bisect start
git bisect bad
git bisect good <commit>
```

Binary-searches through history to find which commit introduced a bug. Git checks out a middle commit each time; you test it and tell Git `good` or `bad`, and it narrows down the culprit.

---

12. Tags (marking releases)

```bash
git tag v1.0.0
```

Creates a lightweight tag on the current commit — usually used for release versions.

```bash
git tag -a v1.0.0 -m "Release message"
```

Creates an annotated tag (stores message, author, date — preferred for real releases).

```bash
git push origin v1.0.0
```

Pushes a single tag to the remote (tags aren't pushed automatically with regular commits).

```bash
git push origin --tags
```

Pushes all local tags at once.

---

13. Quick Reference: "I want to..."

Goal	Command
See what changed	`git status` / `git diff`
Save my work	`git add .` → `git commit -m "..."`
Switch tasks without committing	`git stash`
Create a new feature branch	`git switch -c feature-name`
Get latest changes from remote	`git pull`
Combine my branch into main	switch to `main`, then `git merge feature-name`
Undo my last commit but keep changes	`git reset --soft HEAD~1`
Throw away all local changes	`git reset --hard HEAD`
Undo a commit that's already pushed	`git revert <hash>`
Fix my last commit message	`git commit --amend`
Clean up messy commits before a PR	`git rebase -i HEAD~N`
See who wrote a line	`git blame <file>`
Find which commit broke something	`git bisect start`

---

14. A Few Habits Worth Building

- **Commit often, in small logical chunks** — easier to review, revert, and understand later.
- **Write commit messages in the imperative**: "Add login validation" not "Added" or "Adds."
- **Pull before you push** — reduces conflicts.
- **Never force-push to** `main` / `master` or any shared branch unless you know exactly why and everyone's aware.

- **Use `git status` obsessively** — it costs nothing and prevents most "wait, what did I just do" moments.
- Before a `--hard` reset or `git clean`, run `git status` and/or the `-n` (dry run) version first if one exists — these are the two easiest ways to lose work.

Keep this open while you practice — try running through the "core loop" (status → add → commit → log) a few times, then branch/merge, then try breaking something on purpose and undoing it. That's the fastest way to make it stick.